

# GraphQL

GraphQL: concept, Quality Engineering use, first steps and official reference.

## What is it?

GraphQL is part of the modern software engineering and quality ecosystem. This guide explains its role, where it fits and how QA professionals can use it with context.

## Official site and documentation

Use the official GraphQL source as the primary reference: <https://graphql.org/>. Official documentation should be the first source for installation, supported versions, syntax changes and new features.

## QA use

For QA, GraphQL may support validation, automation, diagnostics, data, integration or operational quality depending on the domain. Focus on the problem it solves and the risks it helps expose.

## First steps

Start with the official documentation. Confirm prerequisites, supported versions, permissions and environment policies before installing or configuring it.

## Good practices

Use controlled environments, keep credentials out of code, capture evidence and version configuration when appropriate.

## Next step

Practice with a small reproducible scenario before adopting the technology in a corporate suite or pipeline.

## When to use it

Use this knowledge when product risk, architecture or the delivery stage requires this type of validation. Confirm the goal, environment, required data and expected evidence before starting.

## How to start safely

Start from the official GraphQL site and documentation. Confirm prerequisites, supported versions, operating system and required permissions. Learn in a controlled environment before using it in a critical workflow.

## First QA-oriented use

Pick one small, verifiable task in GraphQL, configure only what is necessary, run a simple example and confirm the outcome. Repeat with different data or conditions before adding abstractions.

## Using it in a real project

Define where GraphQL fits, who owns configuration, how credentials and data are handled and how results are shared. Integrate it into the team workflow instead of creating an isolated activity.

## Adoption good practices

Standardize versions and configuration, document important decisions, keep secrets out of code and preserve reproducible examples. Reassess whether GraphQL still delivers value as the product evolves.

## Common issues and troubleshooting

When GraphQL fails, verify versions, dependencies, permissions, connectivity, data and environment before assuming a product defect. Preserve the original error and reduce the issue to the smallest reproducible case.